

# PatternShop — Capstone Grading Rubric

## Contents

|      |                                                      |   |
|------|------------------------------------------------------|---|
| 1    | How grading works .....                              | 1 |
| 2    | Automatic fail conditions .....                      | 1 |
| 3    | Criteria and weights .....                           | 2 |
| 4    | Level descriptors .....                              | 2 |
| 4.1  | C1 — Student SPEC.md (15) .....                      | 2 |
| 4.2  | C2 — Pattern plan and justification table (10) ..... | 3 |
| 4.3  | C3 — Correctness beyond the gate (10) .....          | 3 |
| 4.4  | C4 — Pattern use (14) .....                          | 3 |
| 4.5  | C5 — Modern Java and concurrency (8) .....           | 4 |
| 4.6  | C6 — Architecture (6) .....                          | 4 |
| 4.7  | C7 — Test quality (7) .....                          | 4 |
| 4.8  | C8 — Code quality and process (5) .....              | 4 |
| 4.9  | C9 — Report (12) .....                               | 5 |
| 4.10 | C10 — Presentation and defence (13) .....            | 5 |
| 5    | Pattern-justification table template .....           | 5 |
| 6    | Grading sheet .....                                  | 6 |

100 points · Brief: [spec.en.md](https://spec.en.md) · Türkçe: [rubric.tr.md](https://rubric.tr.md)

## 1 How grading works

- The rubric has **10 criteria in three groups**. The groups match the syllabus split of the capstone's 40 % of the course grade: specification and design 25 points (10 %), implementation and tests 50 points (20 %), report and defence 25 points (10 %).
- Every criterion has **four levels**: Excellent, Good, Adequate, Insufficient. Each level is defined by things a grader can check in the submission — a count, a command, a file, a section. The grader picks the highest level whose conditions are **all** met and gives exactly the points of that level.
- Insufficient gives its points only if the deliverable exists; a missing deliverable scores 0.
- The graded state is the commit tagged `capstone-final` in the student's fork (W14, before the presentation slot). `SPEC.md` is additionally graded in the version submitted at the W10 milestone (C1).
- Graders run, on JDK 27, from the repository root:  
`./mvnw -q -pl capstone/starter verify, ./mvnw -q -pl capstone/starter test -Pexercises`  
and `./mvnw -q -pl capstone/starter verify -Pcoverage`.

## 2 Automatic fail conditions

If **any** of the following holds on the tagged commit, the implementation part fails: **criteria C3–C8 score 0** (50 points). C1, C2, C9 and C10 are still graded.

| Gate | Condition                                                                                                                                                                         | How it is checked                                                                                                                                                                                                                                                      |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| G1   | The build fails on JDK 27: compilation error, a <code>-Xlint:all -Werror</code> warning, or a red test of the student's own                                                       | <code>./mvnw -q -pl capstone/starter verify</code>                                                                                                                                                                                                                     |
| G2   | Any of the 83 given acceptance tests is red (failure, error or not run)                                                                                                           | <code>./mvnw -q -pl capstone/starter test -Pexercises</code>                                                                                                                                                                                                           |
| G3   | Any of the 7 given architecture rules is red                                                                                                                                      | ShopArchitectureTest in the verify run                                                                                                                                                                                                                                 |
| G4   | A given test, fixture, test resource, architecture rule or GIVEN API type was modified, deleted, disabled ( <code>@Disabled</code> , removed tag, Surefire exclusion) or weakened | <code>git diff</code> against the published starter for <code>capstone/starter/src/test/**/acceptance/**</code> , the <code>*ExerciseTest</code> and ShopArchitectureTest bindings, <code>src/test/resources/acceptance/**</code> and <code>.../capstone/api/**</code> |

The only allowed build change is the one extension E10 needs (`--enable-preview` isolated as in m10); all given tests must still pass without preview. Academic-integrity violations are handled separately (brief: [Academic integrity and AI assistants](#)).

### 3 Criteria and weights

| Group                    | Id  | Criterion                                        | Points     |
|--------------------------|-----|--------------------------------------------------|------------|
| Specification and design | C1  | Student SPEC.md                                  | 15         |
|                          | C2  | Pattern plan and justification table             | 10         |
| Implementation and tests | C3  | Correctness beyond the gate (extension features) | 10         |
|                          | C4  | Pattern use                                      | 14         |
|                          | C5  | Modern Java and concurrency                      | 8          |
|                          | C6  | Architecture                                     | 6          |
|                          | C7  | Test quality                                     | 7          |
|                          | C8  | Code quality and process                         | 5          |
| Report and defence       | C9  | Report                                           | 12         |
|                          | C10 | Presentation and defence                         | 13         |
| <b>Total</b>             |     |                                                  | <b>100</b> |

## 4 Level descriptors

### 4.1 C1 — Student SPEC.md (15)

Checklist (W10 version for a–g, final version for h): (a) all ten template sections present; (b) every mandatory feature F1–F11 mapped to its acceptance suite; (c) each extension feature has at least three Given / When / Then acceptance criteria and named test classes; (d) a Mermaid domain class diagram and a hexagon diagram; (e) a pattern plan with at least ten rows; (f) at least three boundaries or assumptions where the brief is silent; (g) a week-by-week plan for W10–W13; (h) the final SPEC.md matches the code — every planned pattern and extension exists, or the change log says why it changed.

| Level        | Points | Descriptor                                                  |
|--------------|--------|-------------------------------------------------------------|
| Excellent    | 15     | Submitted by the W10 deadline; all of a–h                   |
| Good         | 11     | Submitted by the W10 deadline; six or seven of a–h          |
| Adequate     | 7      | Four or five of a–h, or submitted up to one week late       |
| Insufficient | 3      | Three or fewer of a–h, or submitted more than one week late |

## 4.2 C2 — Pattern plan and justification table (10)

A row is **complete** when it names (1) a force or problem specific to PatternShop, (2) the participant types, which exist in the code, (3) an alternative considered and why it was not chosen, and (4) a test that shows the pattern working. Table template: section 5.

| Level        | Points | Descriptor                                                                                     |
|--------------|--------|------------------------------------------------------------------------------------------------|
| Excellent    | 10     | At least 10 complete rows, and at least one pattern deliberately <i>not</i> used with a reason |
| Good         | 7      | At least 10 rows, at least 8 of them complete                                                  |
| Adequate     | 5      | At least 10 rows with at least 5 complete, or 8–9 rows all complete                            |
| Insufficient | 2      | Fewer than 8 complete rows                                                                     |

## 4.3 C3 — Correctness beyond the gate (10)

The gate (G2) already requires all given acceptance tests to pass. C3 grades the **extension features** (two; pairs three). An extension is **complete** when every acceptance criterion written for it in `SPEC.md` has at least one green test that names it (in the test name or `@DisplayName`) and the feature is reachable from the CLI.

| Level        | Points | Descriptor                                                                                  |
|--------------|--------|---------------------------------------------------------------------------------------------|
| Excellent    | 10     | All extensions complete                                                                     |
| Good         | 7      | All extensions implemented; at least 75 % of their criteria have green tests                |
| Adequate     | 5      | One extension complete, or all extensions with at least 50 % of their criteria tested green |
| Insufficient | 2      | No extension reaches 50 %                                                                   |

## 4.4 C4 — Pattern use (14)

For every pattern declared with `@PatternRole` and counted (creational, structural, behavioural, concurrency), the grader checks: the annotated types play the roles the module lesson gives for that pattern, and the pattern runs on a path covered by a green acceptance or unit test. A pattern meeting both is **valid**. The **mix** is at least 2 creational, 2 structural, 3 behavioural and 1 concurrency pattern (not Immutable Object).

| Level        | Points | Descriptor                                                                                |
|--------------|--------|-------------------------------------------------------------------------------------------|
| Excellent    | 14     | At least 12 valid patterns, mix met by valid patterns, and no declared pattern is invalid |
| Good         | 10     | 10 or 11 valid patterns, mix met by valid patterns                                        |
| Adequate     | 7      | 8 or 9 valid patterns, or at least 10 valid patterns without the mix                      |
| Insufficient | 3      | 7 or fewer valid patterns                                                                 |

## 4.5 C5 — Modern Java and concurrency (8)

Checklist: (a) at least one sealed hierarchy of records handled with an exhaustive `switch` with record patterns and no `default`; (b) at least two further sealed hierarchies or exhaustive switches in the student's own code; (c) all value types are records (no mutable value classes); (d) single-method strategies or commands are lambdas or method references where no state is needed; (e) fulfilment runs on virtual threads with a bounded number of parallel orders, and `SPEC.md` §6 argues why shared state is thread-safe; (f) `Optional` only as a return type; no public method returns `null`.

| Level        | Points | Descriptor                                  |
|--------------|--------|---------------------------------------------|
| Excellent    | 8      | All of a–f                                  |
| Good         | 6      | Five of a–f                                 |
| Adequate     | 4      | Three or four of a–f, including (a) and (e) |
| Insufficient | 2      | Otherwise                                   |

## 4.6 C6 — Architecture (6)

The gate (G3) already requires the given rules to be green. Checklist: (a) adapters are constructed only in the composition root ( `config` ); (b) outbound ports are interfaces owned by `application` and named after their role, not a technology; (c) business outcomes are sealed results, infrastructure failures are exceptions; (d) at least one additional ArchUnit rule of the student's own, green and meaningful (for example: ports are interfaces, naming of adapters); (e) the hexagon diagram in `SPEC.md` matches the actual packages.

| Level        | Points | Descriptor          |
|--------------|--------|---------------------|
| Excellent    | 6      | All of a–e          |
| Good         | 4      | Four of a–e         |
| Adequate     | 3      | Three of a–e        |
| Insufficient | 1      | Two or fewer of a–e |

## 4.7 C7 — Test quality (7)

Checklist: (a) every valid pattern (C4) has at least one own unit test that shows its behaviour; (b) test doubles are hand-written; no mocking library; (c) test names describe behaviour ( `appliesCouponAfterThresholdDiscount`, not `test1` ); (d) tests are deterministic: no `Thread.sleep` for synchronisation, time comes from an injected clock; (e) line coverage of `domain` and `application` is at least 80 % (JaCoCo report of `-Pcoverage` ); (f) every extension test is listed next to its acceptance criterion in `SPEC.md`.

| Level        | Points | Descriptor           |
|--------------|--------|----------------------|
| Excellent    | 7      | All of a–f           |
| Good         | 5      | Five of a–f          |
| Adequate     | 3      | Three or four of a–f |
| Insufficient | 1      | Two or fewer of a–f  |

## 4.8 C8 — Code quality and process (5)

Checklist: (a) no `@SuppressWarnings` without a comment explaining why; (b) every public type in `domain` and `application` has a one-line Javadoc of intent; (c) no class has more than 10 public methods and no

method is longer than 30 lines; (d) at least one commit in each of the weeks W9–W13, with Conventional Commit messages; (e) no leftover `TODO(capstone)` markers, no commented-out code, no unused classes.

| Level        | Points | Descriptor          |
|--------------|--------|---------------------|
| Excellent    | 5      | All of a–e          |
| Good         | 4      | Four of a–e         |
| Adequate     | 2      | Three of a–e        |
| Insufficient | 1      | Two or fewer of a–e |

#### 4.9 C9 — Report (12)

Checklist for `REPORT.md`: (a) a design overview with the hexagon diagram; (b) the complete pattern-justification table (may be the same as in `SPEC.md`, updated); (c) at least two modern-Java decisions explained against the classic alternative; (d) the concurrency design with its thread-safety argument; (e) a reflection: at least two things you would change, and one pattern you deliberately did not use and why; (f) an AI-usage statement (what for, which parts); (g) a one-page summary in the other language (English ↔ Turkish); (h) 6–10 pages, terminology as in `docs/glossary.md`, every claim about the code points to a file path.

| Level        | Points | Descriptor            |
|--------------|--------|-----------------------|
| Excellent    | 12     | All of a–h            |
| Good         | 9      | Six or seven of a–h   |
| Adequate     | 6      | Four or five of a–h   |
| Insufficient | 2      | Three or fewer of a–h |

#### 4.10 C10 — Presentation and defence (13)

Checklist: (a) the talk takes 9–11 minutes; (b) a live CLI demo of checkout, fulfilment and one report works; (c) three patterns are explained with their force, structure and the student's code; (d) one trade-off is presented with the alternative that was rejected. Defence: the examiner asks four questions about the student's own code — locate the participants of a named pattern, explain what a named test proves, sketch how a given change request would be implemented, explain why fulfilment is thread-safe. Pairs: each member presents a part and answers at least two of the questions.

| Level        | Points | Descriptor                                                                          |
|--------------|--------|-------------------------------------------------------------------------------------|
| Excellent    | 13     | All of a–d, and four of four questions answered correctly                           |
| Good         | 10     | At least three of a–d including (b), and three of four questions answered correctly |
| Adequate     | 6      | At least two of a–d, and two of four questions answered correctly                   |
| Insufficient | 3      | Otherwise (presentation given)                                                      |

## 5 Pattern-justification table template

Copy this table into `SPEC.md` (§5) and `REPORT.md`. One row per counted pattern; add rows for architectural patterns if you like (they do not count towards the ten). The first row is an example of the expected depth.

| #   | Pattern (category)     | Force / problem in Pattern-Shop                                                                                     | Participants (types and @PatternFole roles)                                                                              | Alternative considered and why not                                                                                       | Modern Java form                                                             | Test(s) that show it                                                                                     | Course module |
|-----|------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|---------------|
| 1   | Strategy (behavioural) | Four promotion kinds must be priced the same way, and new kinds will be added without touching the pricing pipeline | PromotionRule (strategy), BuyXGetYFreeRule, CategoryPercentOffRule, ... (concrete strategies), PricingPipeline (context) | switch over the promotion spec inside the pipeline: shorter today, but every new kind changes the pipeline and its tests | sealed interface of records; lambdas not used because each rule carries data | PricingAcceptance.categoryPercentOffRoundHalfEvenPerLine, PromotionRuleTest.buyXGetYCountsWholeGroupOnly | m06           |
| 2   |                        |                                                                                                                     |                                                                                                                          |                                                                                                                          |                                                                              |                                                                                                          |               |
| ... |                        |                                                                                                                     |                                                                                                                          |                                                                                                                          |                                                                              |                                                                                                          |               |

**Patterns considered and rejected** (at least one for the Excellent level in C2):

| Pattern | Where it was tempting | Why it was not used |
|---------|-----------------------|---------------------|
|---------|-----------------------|---------------------|

## 6 Grading sheet

| Id    | Criterion                                   | Max        | Level | Points | Evidence | (files, commands, notes) |
|-------|---------------------------------------------|------------|-------|--------|----------|--------------------------|
| G1–G4 | Gates passed? (yes / no — if no, C3–C8 = 0) | —          |       |        |          |                          |
| C1    | Student SPEC.md                             | 15         |       |        |          |                          |
| C2    | Pattern plan and justification table        | 10         |       |        |          |                          |
| C3    | Correctness beyond the gate                 | 10         |       |        |          |                          |
| C4    | Pattern use                                 | 14         |       |        |          |                          |
| C5    | Modern Java and concurrency                 | 8          |       |        |          |                          |
| C6    | Architecture                                | 6          |       |        |          |                          |
| C7    | Test quality                                | 7          |       |        |          |                          |
| C8    | Code quality and process                    | 5          |       |        |          |                          |
| C9    | Report                                      | 12         |       |        |          |                          |
| C10   | Presentation and defence                    | 13         |       |        |          |                          |
|       | <b>Total</b>                                | <b>100</b> |       |        |          |                          |